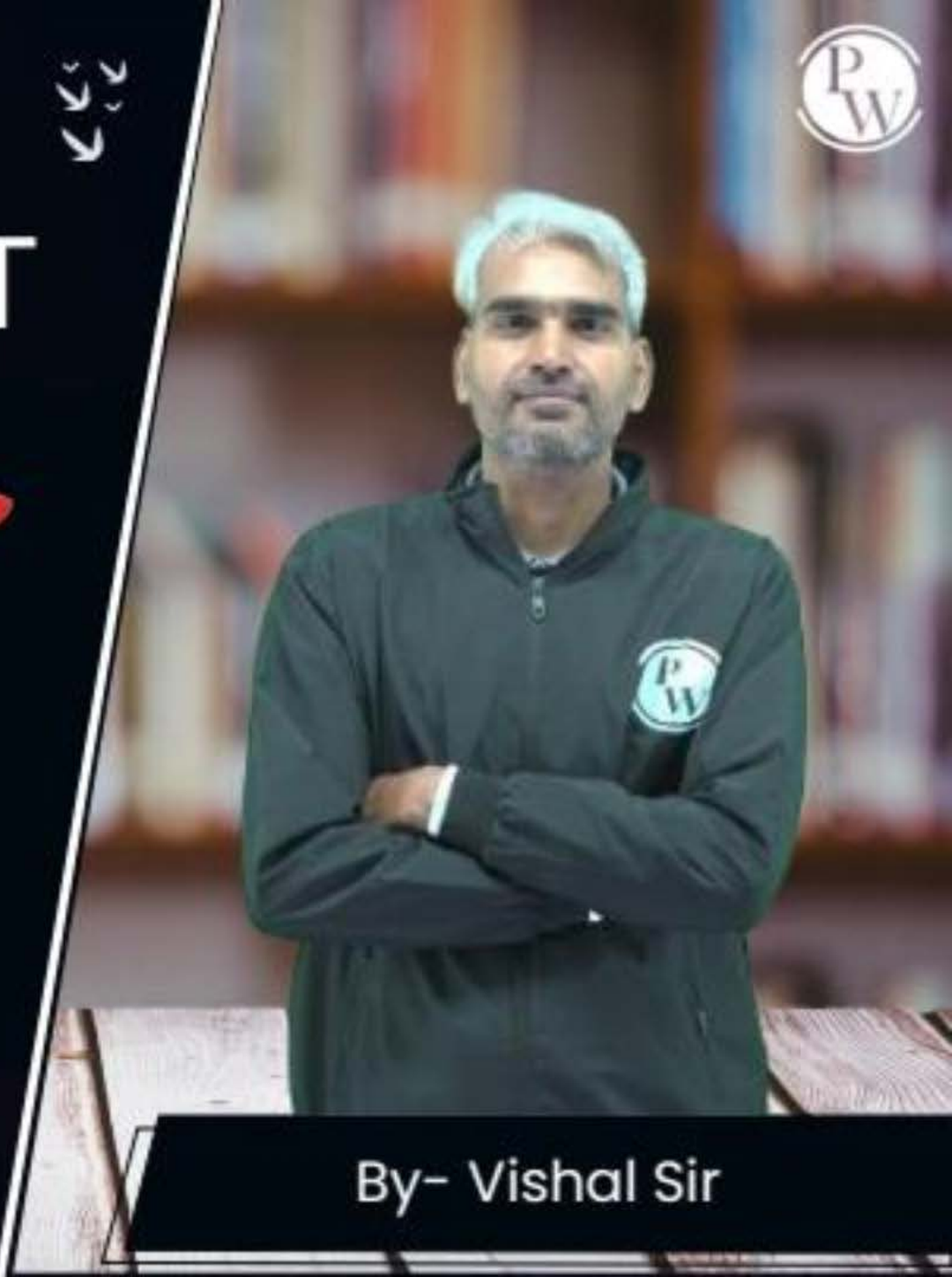
Computer Science & IT



Database Management System

File organization And Indexing

Lecture No. 06



By- Vishal Sir

Recap of Previous Lecture



- Topic: Balanced multiway search tree
- Topic: B tree

Slide

Topics to be Covered



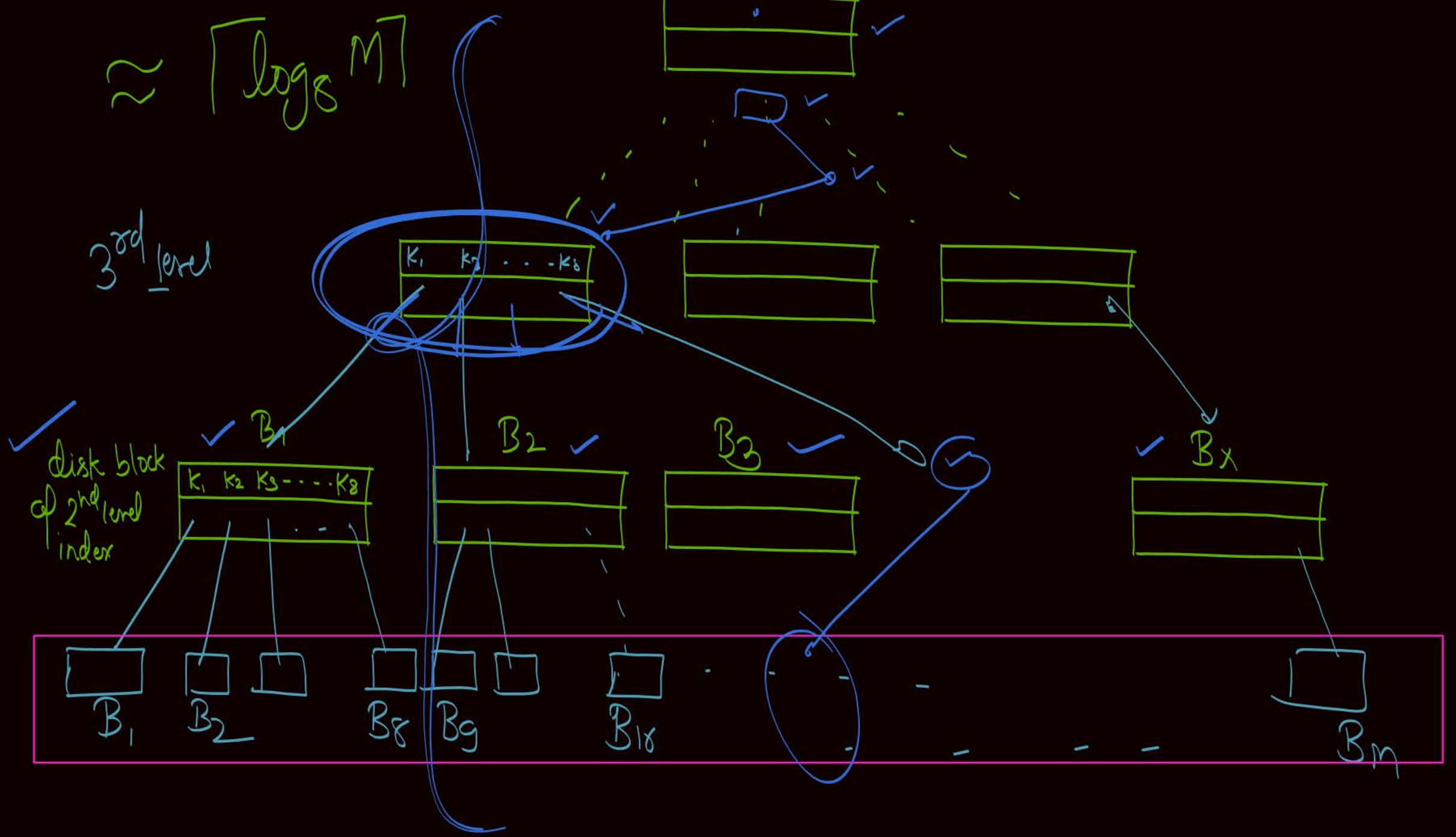
- Topic: B+ tree

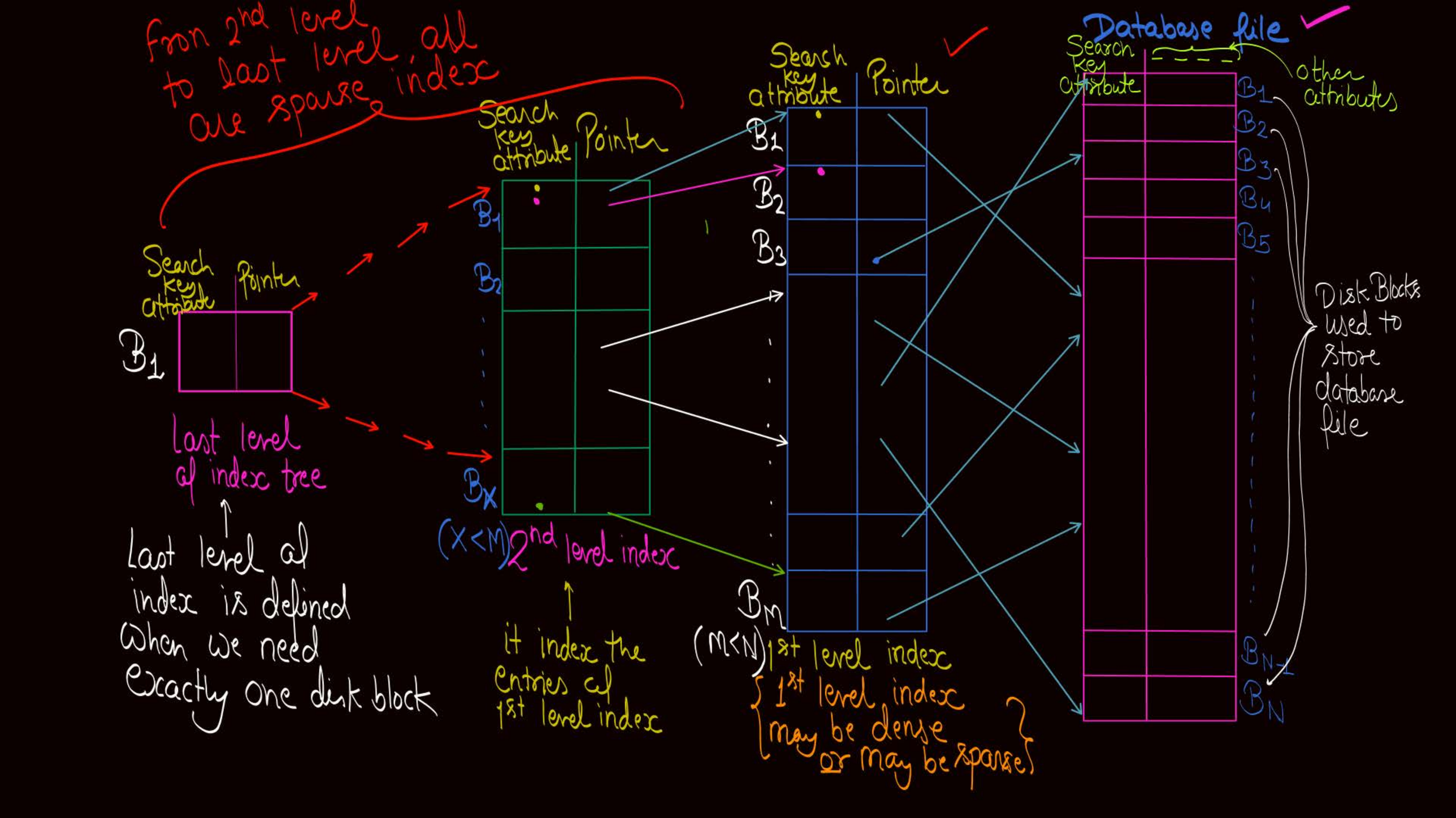
Slide



Topic : Multi-level Index

* Multi-level index is used to reduce the IO Cost w.r.t. single level index


≃ ⌈log8 M⌉
3rd level
K1 K2 . . . K8
B1
B2
B3
Bx
disk block of 2nd level index
K1 K2 K3 - - - - K8
B1
B2
B8
B9
B18
Bm


From 2nd level to last level, all are sparse index
Search key attribute
Pointer
B1
Last level of index tree
Last level of index is defined when we need exactly one disk block
Search key attribute
Pointer
B1
B2
Bx
(x<M) 2nd level index
it index the entries of 1st level index
Search key attribute
Pointer
B1
B2
B3
BM
(M<N) 1st level index
{ 1st level index may be dense or may be sparse }
Database file
Search key attribute
other attributes
B1
B2
B3
B4
B5
BN-1
BN
Disk Blocks used to store database file

Note :→

IO Cost using multi-level index

IO Cost using multi-level Index = Number of levels in multi-level index + 1

Number of levels in multi-level index: To locate an entry in the 1st level index of multi-level index

+ 1: For a block of database file to access corresponding record

Note :→

IO Cost using multi-level index

IO Cost using multi-level Index = Number of levels in multi-level index + 1

it is same for all access

i.e, same IO Cost for all three cases

Best Case, Worst Case, Average Case



Topic : Multi-level Index

In multi-level index,

- (1) 1st level of index may be dense or may be sparse
- (2) Except first level, all other levels of index are always sparse index

* Multi-level index is suitable only for static database
  i.e. the database in which insertion & deletion are very rare

* But if database is dynamic {i.e. insert and delete opⁿ are frequent}
  then after every insertion and/or deletion we may have to restructure all the levels of multi-level index. and it will be a time consuming operation

  ∴ for dynamic database we use

  B tree or B+ tree

  these are self Balancing multi-way search tree

* Self-balancing multi-way search tree

B tree
B+ tree

These are balanced, multi-way search tree

all leaf nodes are at same level

↓

m-ary search tree



Topic : B Tree

Order of a node of B tree

In general, order of a node of B tree is defined as the maximum number of child pointer a node can have

Consider a B tree of order = p ,

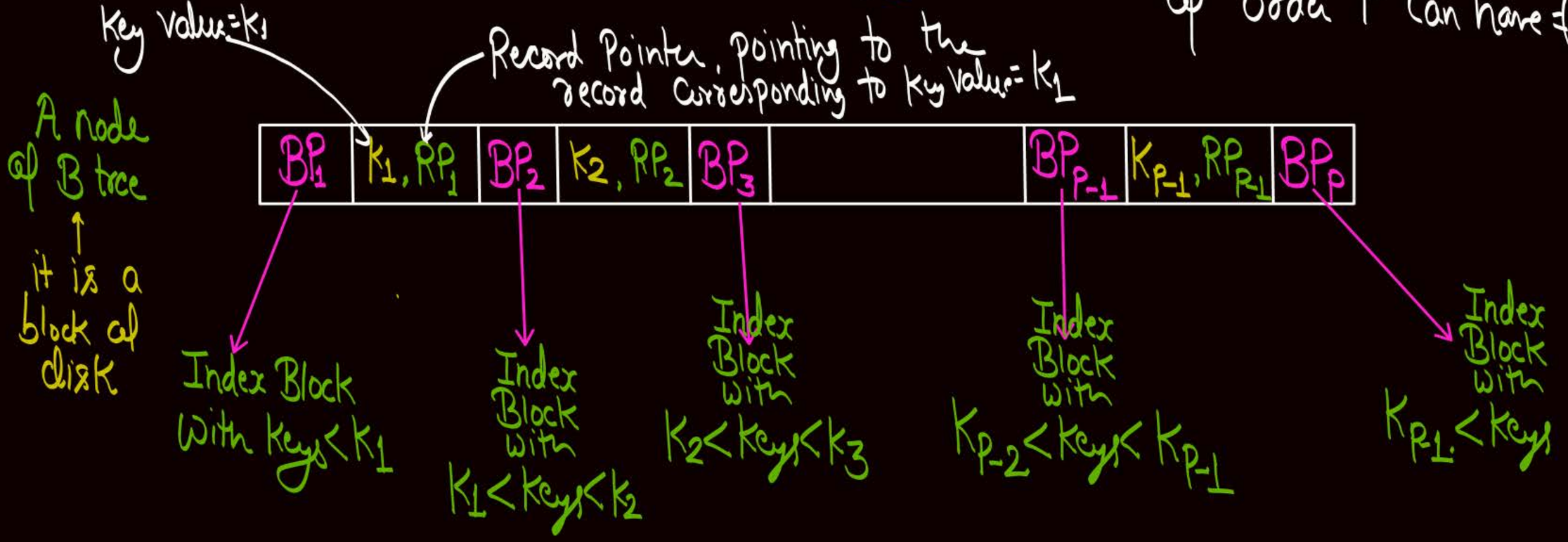
↳ i.e: maximum number of Child pointer a node can have = p

∴ Maximum number of keys a node of B tree of order 'p' can have = (p-1)

Structure of an internal node of B tree

All keys within the node will always be stored in ascending order

↳ i.e. $K_1 < K_2 < K_3 - - - < K_{p-1}$



Consider a B tree of order = p ,

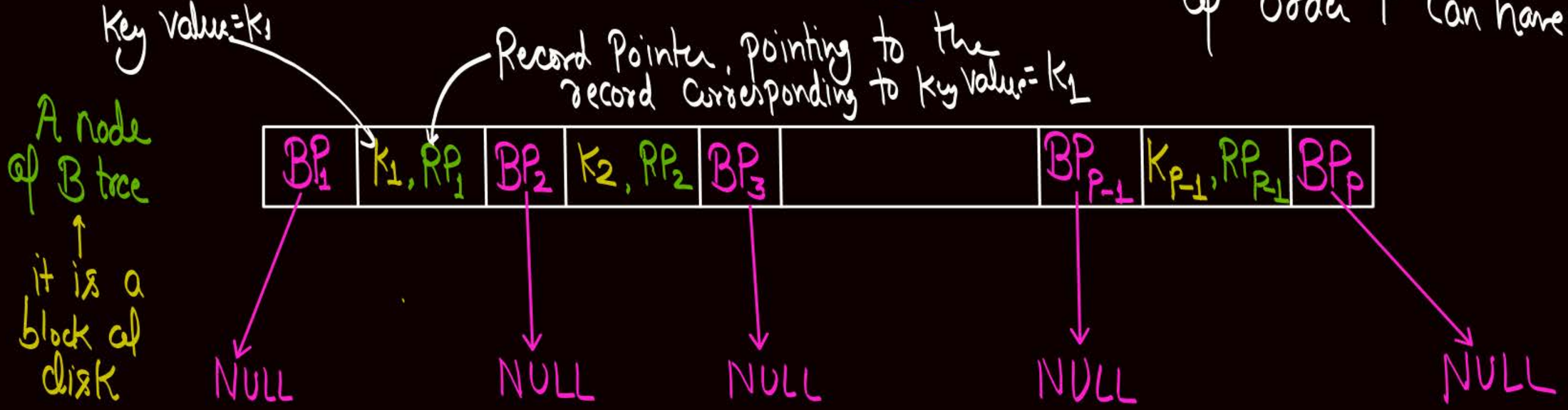
↳ i.e. maximum number of Child pointer a node can have = p

∴ Maximum number of keys a node of B tree of order 'p' can have = (p-1)

Structure of leaf node of B tree

All keys within the node will always be stored in ascending order

↳ i.e. $K_1 < K_2 < K_3 - - - < K_{p-1}$



- BP (Block Pointer) : It is a pointer that points to the index block at lower level of B tree

  { Child Pointer
    node Pointer
    Tree Pointer }

- RP (Record Pointer) : It Points to the record in the database file corresponding the associated key value

* Note :- In B tree, record pointers are present in internal node as well as in leaf node.

∴ Structure of B tree is different from the structure of multi-level index

* In multi-level index record pointers are present only at leaf level {i.e. 1st level index}

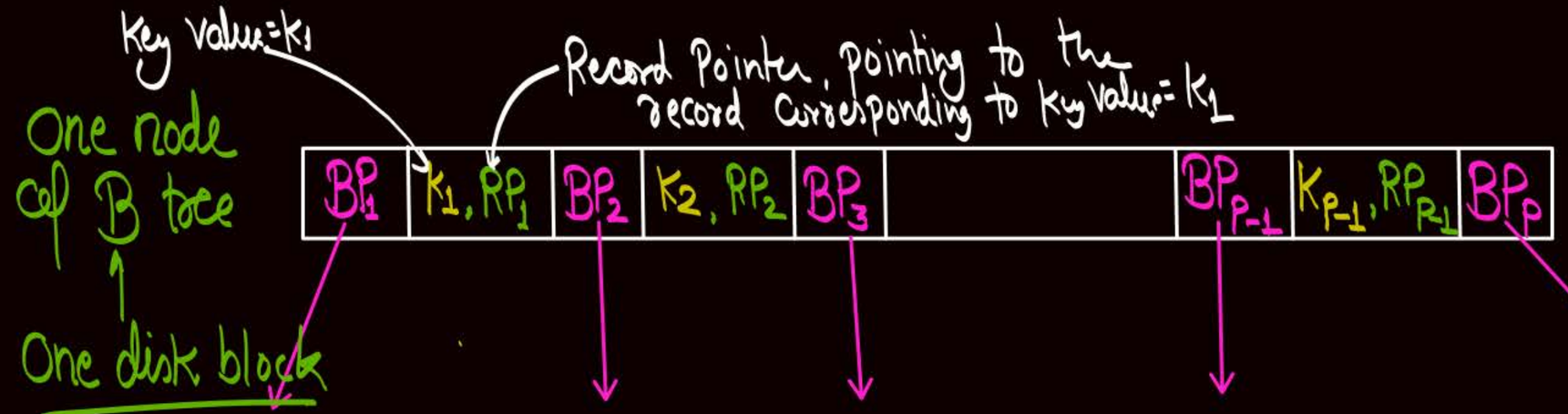
Note:- Each node of B tree is a disk block

∴ whatever information we want to store in one node of B tree must fit in a single disk block.

Consider a B tree of order = p ,

↳ i.e. maximum number of Child Pointer a node can have = p

∴ Maximum number of Keys a node of B tree of order 'p' can have = (p-1)



$$\left[\left(p * \text{Block Pointer Size}\right) + (p-1) * \left(\text{Key Field Size} + \text{Record Pointer Size}\right)\right] \leq \text{Disk block Size}$$

Using this equation we can obtain maximum order possible for a node of B tree



Topic : B Tree

Consider a relations R with a key filed K. A B-tree of order P is used to access structures on K. Where 'P' denotes the maximum Number of tree pointers in a B-tree index node. Assume that key (k) is 30 bytes long disk block size 1024 byte. Each data (Record Pointer) pointer size P_D is 12 bytes long & each block pointer P_B is 10 byte long. In order for each B-Tree Node to fit in a single disk block, the maximum value of P is-

$$(P * BP_{size}) + (P-1)(Key_{size} + RP_{size}) \leq \text{Disk Block Size}$$

$$P * 10 + (P-1)(30+12) \leq 1024$$

$$52P \leq 1024+42$$

$$P \leq 20.5$$

$$\Rightarrow P \leq 20 \Rightarrow P_{max} = 20$$

A 19

B 20

C 21

D 22

Important

Consider a B tree of order = p

- Maximum number of child pointer a node can have is = p
- Maximum number of keys a node can have is = p-1
- Minimum number of child pointer a non-root node must have is = $\lceil p/2 \rceil$
- Minimum number of keys a non-root node must have is = $\lceil p/2 \rceil - 1$
- Minimum number of child pointer a root node must have is = 2
- Minimum number of keys root node must have is = 1

Each non-root node must be at least half filled

#Q. a) Find the minimum number of nodes required in B tree of order = p and height = h

Consider, root is at height=0

b) Find the minimum number of child pointers required in B tree of order =p and height = h

c) Find the minimum number of keys required in B tree of order=p and height = h

d) Find the maximum number of nodes possible in B tree of order = p and height = h

e) Find the maximum number of child pointers possible in B tree of order = p and height = h

f) Find the maximum number of keys possible in B tree of order = p and height = h

Analysis w.r.t. B tree of order = p

* H.W.

Height/level	Minimum number of nodes at	Minimum number of Child Pointers at	Minimum number of Keys/RP at	Maximum number of nodes at	Maximum number of Child Pointers at	Maximum number of Keys/RP at
h = 0 / l = 1	1 (root)	2	1	1	p	(p-1)
h = 1 / l = 2	2 (non-root)	$2 * \lceil \frac{p}{2} \rceil$	$2 * (\lceil \frac{p}{2} \rceil - 1)$	p	$p * p = p^2$	$p(p-1)$
h = 2 / l = 3	$2 * \lceil \frac{p}{2} \rceil$	$2 * \lceil \frac{p}{2} \rceil * \lceil \frac{p}{2} \rceil = 2 * \lceil \frac{p}{2} \rceil^2$	$2 * \lceil \frac{p}{2} \rceil * (\lceil \frac{p}{2} \rceil - 1)$	p^2	$p^2 * p = p^3$	$p^2(p-1)$
h = 3 / l = 4	$2 * \lceil \frac{p}{2} \rceil^2$	$2 * \lceil \frac{p}{2} \rceil^2 * \lceil \frac{p}{2} \rceil = 2 * \lceil \frac{p}{2} \rceil^3$	$2 * \lceil \frac{p}{2} \rceil^2 * (\lceil \frac{p}{2} \rceil - 1)$	p^3	p^4	$p^3(p-1)$
⋮	⋮	⋮	⋮	⋮	⋮	⋮
Height = h	$2 * \lceil \frac{p}{2} \rceil^{h-1}$	$2 * \lceil \frac{p}{2} \rceil^h$	$2 * \lceil \frac{p}{2} \rceil^{h-1} * (\lceil \frac{p}{2} \rceil - 1)$	p^h	p^{h+1}	$p^h(p-1)$

Summation of GP : $\frac{a(r^n - 1)}{(r-1)}$

a : first term

r = Common ratio

n = total no. of terms

Minimum number of nodes in a B tree of height = h and order = p

$$1 + 2 + 2 * \lceil \frac{p}{2} \rceil + 2 \lceil \frac{p}{2} \rceil^2 + \text{- - - -} + 2 * \lceil \frac{p}{2} \rceil^{h-1}$$

$$1 + 2 \lceil \frac{p}{2} \rceil^0 + 2 * \lceil \frac{p}{2} \rceil^1 + 2 \lceil \frac{p}{2} \rceil^2 + \text{- - - -} + 2 * \lceil \frac{p}{2} \rceil^{h-1}$$

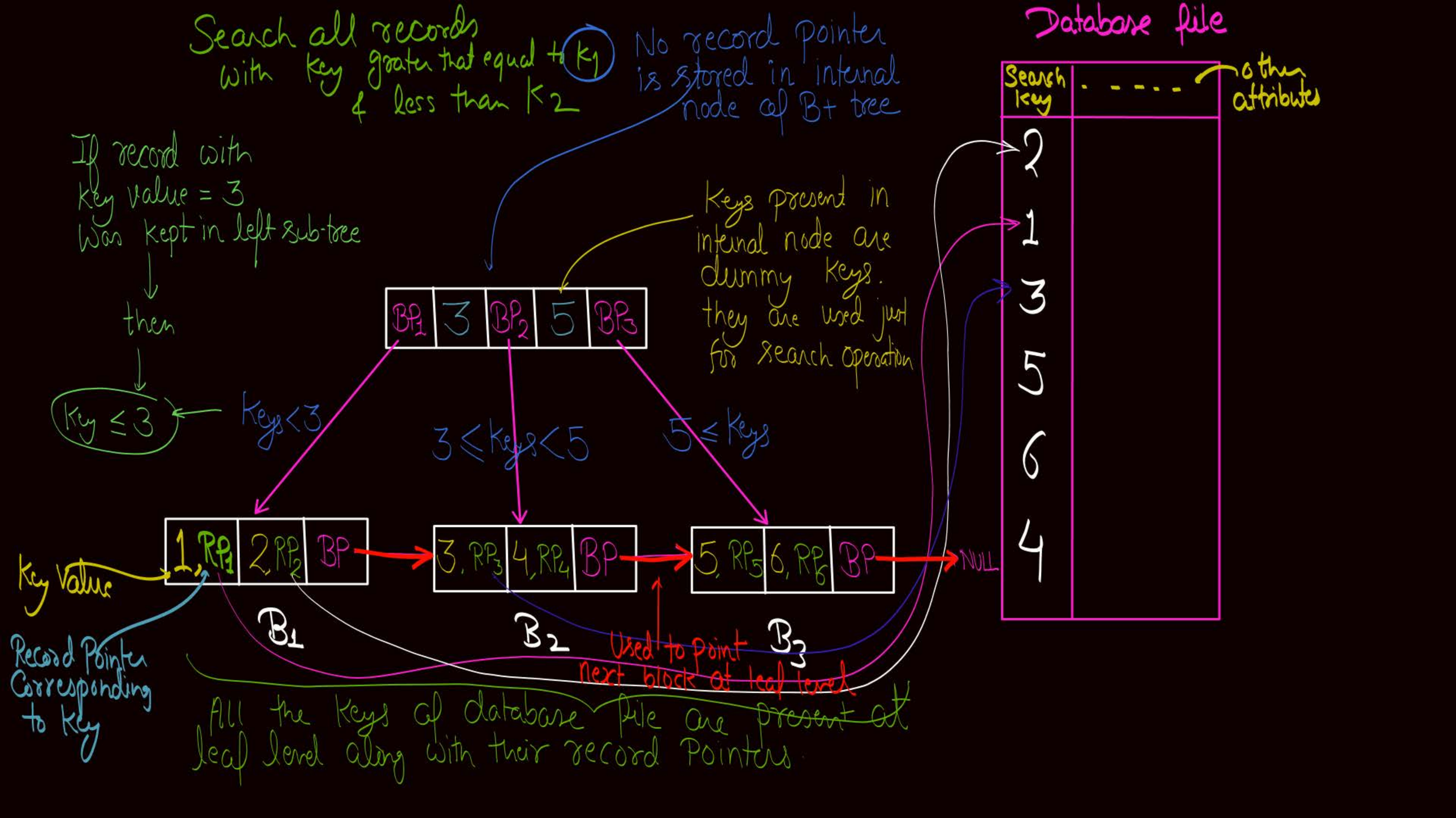
$$1 + \left(2 \lceil \frac{p}{2} \rceil^0 + 2 * \lceil \frac{p}{2} \rceil^1 + 2 \lceil \frac{p}{2} \rceil^2 + \text{- - - -} + 2 * \lceil \frac{p}{2} \rceil^{h-1} \right)$$

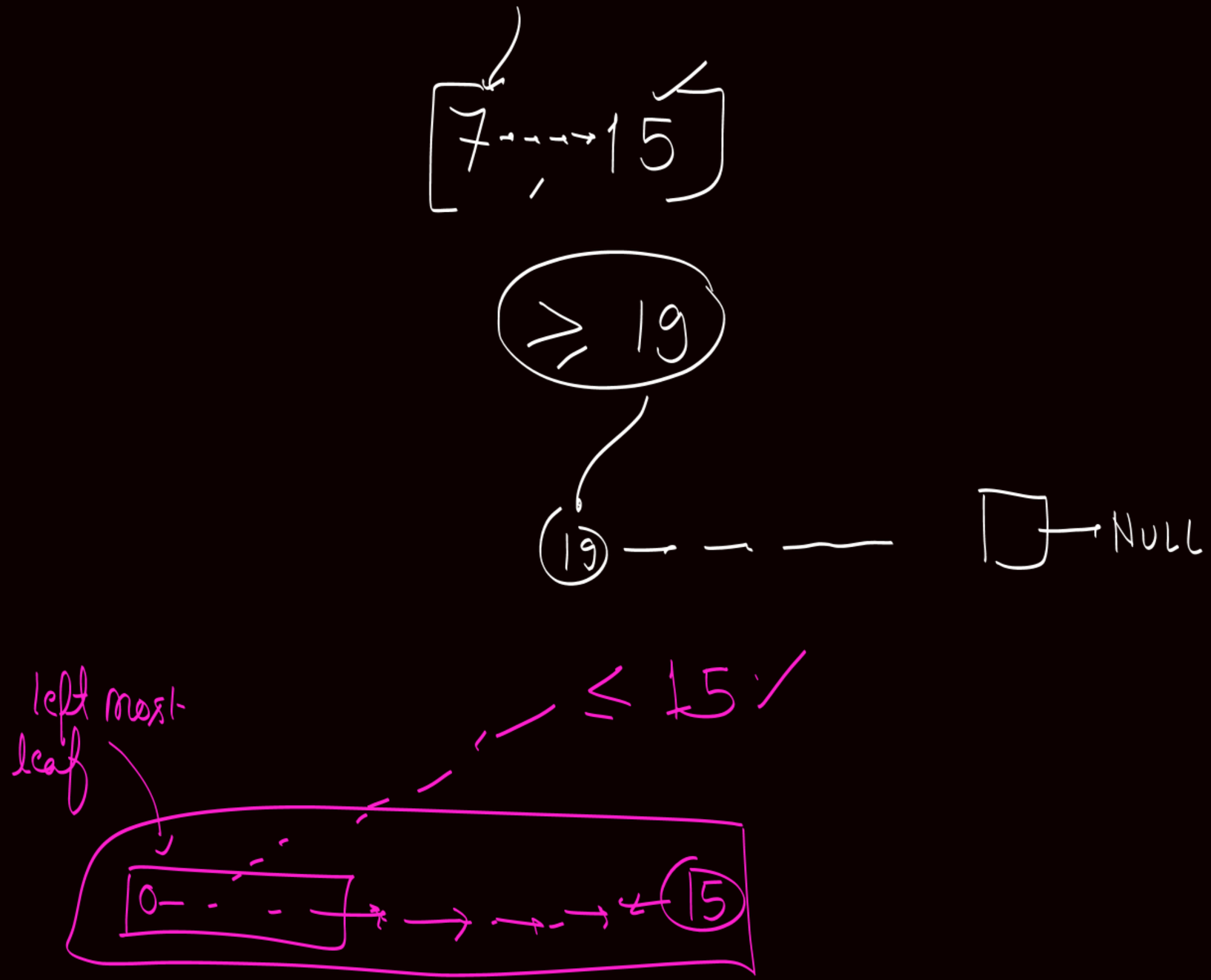
$$1 + \frac{2 * \left(\lceil \frac{p}{2} \rceil^h - 1 \right)}{\left(\lceil \frac{p}{2} \rceil - 1 \right)}$$



Topic : B+ tree visualization

B+ tree is some-what similar to multi-level index


Search all records with key greater that equal to K_1 & less than K_2
No record pointer is stored in internal node of B+ tree
Database file
Search key
other attributes
If record with key value = 3 was kept in left sub-tree
then
Key ≤ 3
Keys < 3
Keys present in internal node are dummy keys. they are used just for search operation
$3 \leq Keys < 5$
$5 \leq Keys$
BP_1 3 BP_2 5 BP_3
1, RP_1 2, RP_2 BP
3, RP_3 4, RP_4 BP
5, RP_5 6, RP_6 BP
NULL
Key Value
Record Pointer Corresponding to Key
B_1
B_2
B_3
Used to point next block at leaf level
All the keys of database file are present at leaf level along with their record Pointers


7 ---→ 15
≥ 19
19
NULL
left most leaf
≤ 15
0
15



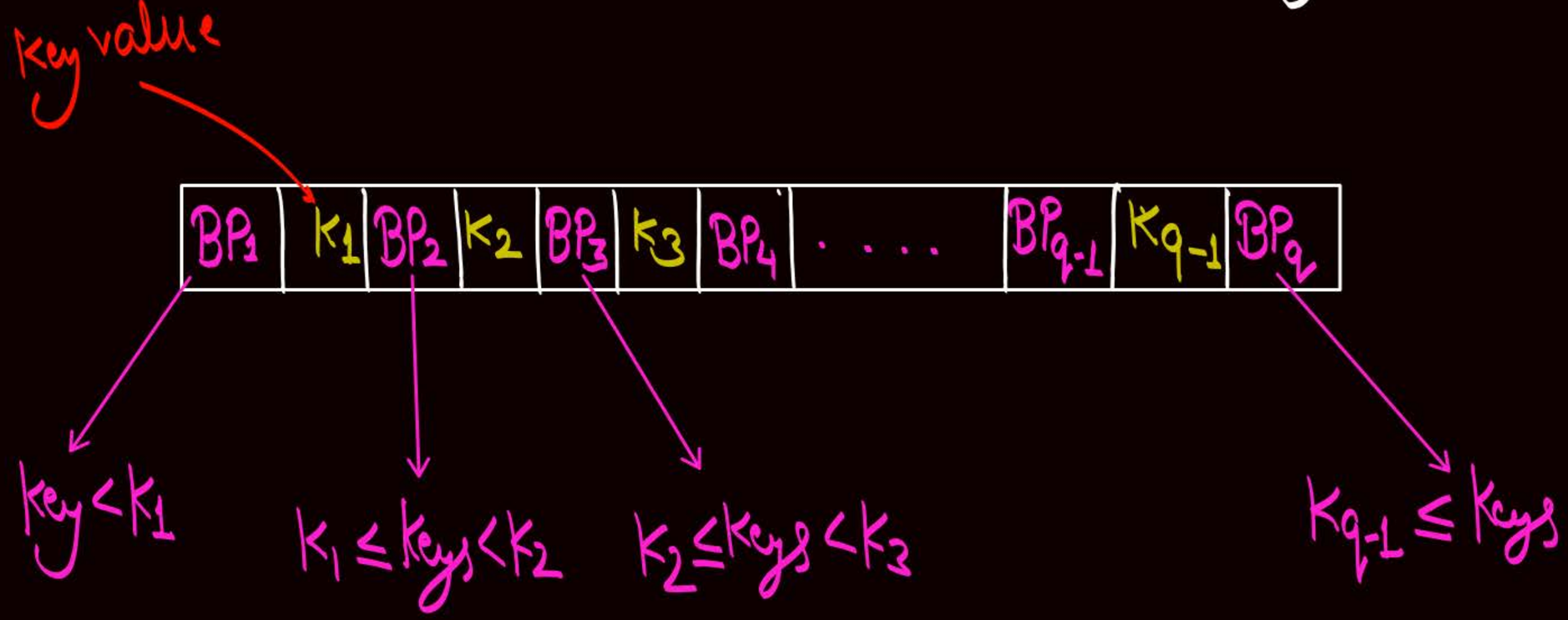
Topic : B+ Tree

- B+ tree is some what similar to normal multi-level index.

- In B+ Tree all keys along with their record pointers are present at leaf nodes of B+ Tree.
- Keys within a node of B+ tree are always stored in ascending order.
- In B+ Tree keys in the internal nodes are used only for search operation (they are dummy keys), they are not used to point any record of the database as no record pointer is available in internal nodes of B+ Tree.

Structure of an internal node of B+ tree

* All keys within the node are in increasing order





Topic : Order of Internal node of B+ Tree

Order of an internal node of B+ tree is defined as the maximum number of child pointer (block pointer/ node pointer) a B+ tree node can have. (same as B Tree)

Let order of internal node of B+ tree is denoted by 'q'

Whatever we want to store in an internal node of B+ tree must fit in a single disk block

∴ If order of internal node of B+ tree is given as "q"

then $$\left[\left(q * \text{Block Pointer Size}\right) + \left((q-1) * \text{Key Field Size}\right)\right] \leq \text{Disk Block Size}$$

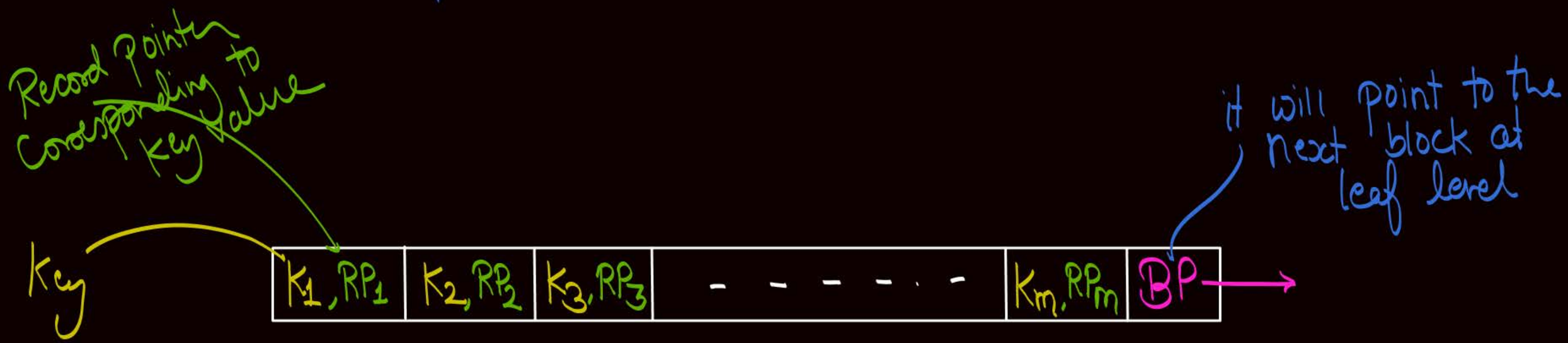


Topic : Order of Internal node of B+ Tree

Let 'q' is the order of an internal node of B+ Tree

- Maximum number of child pointer an internal node can have is = q
- Maximum number of keys an internal node can have is = q-1
- Minimum number of child pointer a non-root internal node must have is = $\lceil q/2 \rceil$
- Minimum number of keys a non-root internal node must have is = $\lceil q/2 \rceil$-1
- Minimum number of child pointer root node must have is = 2
- Minimum number of keys root node must have is = 1

Structure of leaf node of B+ tree





Topic : Order of leaf node of B+ Tree

- Order of leaf a node of B+ tree is defined as the maximum number of keys(record pointers) a B+ tree node can have. (different from B Tree)

Let order of a leaf node of B+ tree is denoted by 'm'

Whatever we want to store in a leaf node of B+ tree must fit in a single disk block

→ If order of leaf node of B+ tree is given as 'm'

then

$$\{m * (\text{Key field size} + \text{Record Pointer size}) + 1 * \text{Block Pointer size}\} \leq \text{Disk block size}$$



Topic : Order of leaf node of B+ Tree

Because node must be at least half filled

Let 'm' is the order of a leaf node of B+ Tree

- Maximum number of keys a leaf node can have is = m
- Minimum number of keys a non-root leaf node must have is = $\lceil m/2 \rceil$
- If leaf node is root node, then it minimum no. of keys = 1
- There will be exactly one block pointer in each leaf node, it will point to the next leaf node of B+ tree. (because of this pointer range query is more efficient in B+ tree)



Topic : NOTE

"w.r.t. B+ tree"

- If only one order is given in the question, then we will use the same order for both leaf node as well as for internal node, but it will be handled differently

Imp:-

Consider order of node of B+ tree is = n { Only one order is given }

Korth

* Maximum number of child Pointer an internal node of B+ tree can have = n
* Maximum number of Keys an internal node of B+ tree can have = $(n-1)$
* Minimum number of child Pointer a non-root internal node of B+ tree must have = $\lceil \frac{n}{2} \rceil$
* Minimum number of Keys a non-root internal node of B+ tree must have = $\left(\lceil \frac{n}{2} \rceil - 1\right)$

for root node it will be '2' & '1' respectively

Navathe

* Maximum number of child Pointer an internal node of B+ tree can have = n
* Maximum number of Keys an internal node of B+ tree can have = $(n-1)$
* Minimum number of child Pointer a non-root internal node of B+ tree must have = $\lceil \frac{n}{2} \rceil$
* Minimum number of Keys a non-root internal node of B+ tree must have = $\left(\lceil \frac{n}{2} \rceil - 1\right)$

for root node it will be '2' & '1' respectively

Imp:-

Consider order of node of B+ tree is = n

{ only one order is given }

Korth

* Maximum number of key a leaf node of B+ tree can have $= (n-1)$

* Minimum number of keys a non-root leaf node must have $= \left\lceil \frac{(n-1)}{2} \right\rceil$

Navathe

* Maximum number of key a leaf node of B+ tree can have $= (n-1)$

* Minimum number of keys a non-root leaf node must have $= \left\lceil \frac{n}{2} \right\rceil$



2 mins Summary

Topic B+ tree

Slide

THANK - YOU